```
 printk("Timer removed");
 return;
}
```

Within *init_module*, *setup_timer* initialises the timer and sets it off by calling *mod_timer*. When the timer expires, *timer1_expires* is called. When the module is removed, the timer is deleted by invoking *del_timer*. Save this in a directory of your choice and name the file *timer_example.c*. The next step will be to create a *Makefile* in the same directory, as shown below:

```
obj-m += timer_example.o
 KDIR := /lib/modules/$(shell uname -r)/build
 PWD := $(shell pwd)

all:
 $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
 $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Now, in your terminal, as the *root*, go to the source code directory and run *make* to compile the code (see Figure 1). After compiling, you will find the output *timer_example.ko* file, which has to be inserted into the kernel, with *insmod timer_example.ko*. Since there are no error messages, how does one verify that the module has been inserted? Check the *dmesg* output with *dmesg | tail* (Figure 2). Now remove the module (*rmmod timer_example*) and the timer will also be removed, as you can see with another *dmesg | tail* (Figure 3). To get rid of the messages with warnings, include a macro *MODULE_LICENSE()* with the string *'GPL'* included in brackets as a parameter.

High-resolution timer API

The simple API discussed above is easy to implement, but is not accurate enough to be used for real-time applications. The high-resolution timer API (*'hrtimer'*) has almost the same structure (below) as the simple API, but some changes make it work with great accuracy. The global *jiffies* is not used, but a special data type *ktime* is used to represent time. This structure defines timers from a user perspective. This is implied by *_softexpires* (the expiry time given when the timer was armed), which gives the absolute earliest expiry time of the *hrtimer*. The *function member* is the same as before; *start_pid* is an important field—it stores the PID of the task that started the timer, and *start_comm* stores the name of the process that started the timer.

```
struct hrtimer {
 struct timerqueue_node node;
 ktime_t _softexpires;
 enum hrtimer_restart (*function)(struct hrtimer *);
 struct hrtimer_clock_base *base;
 unsigned long state;
#ifdef CONFIG_TIMER_STATS
 int start_pid;
 void *start_site;
 char start_comm[16];
#endif
};
```

The *hrtimer* structure must be initialised by *hrtimer_init()*, and then it can be started with *hrtimer_start*. This call includes the expiration time (in *ktime_t*) and the mode of the time value (absolute or relative value). The prototypes are:

```
void hrtimer_init( struct hrtimer *time, clockid_t which_clock,
enum hrtimer_mode mode );
int hrtimer_start(struct hrtimer *timer, ktime_t time, const
enum hrtimer_mode mode);
```

To cancel a timer, use *hrtimer_cancel* or *hrtimer_try_to_cancel*. The first will cancel the timer, but if the timer has already fired, it'll wait for the *callback* function to finish, and then cancel the timer. The latter will return false if the timer has already fired. Use this as per your needs.

```
extern int hrtimer_cancel(struct hrtimer *timer);
extern int hrtimer_try_to_cancel(struct hrtimer *timer);
```

There are even more timer operations, which can be found in *include/linux/hrtimer.h*.

Some points to take care of

Kernel timers are run as a result of software interrupts. The timer function must be atomic in all cases. Kernel timers can cause serious race conditions, even on uni-processor systems, since they are asynchronous with other code. So, any piece of code used by the timer *callback* function must be protected from concurrent access, either from atomic types, or by using spin-locks. Last, but not least, a timer should run on the same CPU that registers it. END

References

[1] *http://en.wikipedia.org/wiki/Kernel_(computing)*
[2] *http://fedoraproject.org/wiki/Building_a_custom_kernel*
[3] *http://www.kernel.org/doc/Documentation/timers/hrtimers.txt*

By: Rahul Gupta

The author graduated from BVCOE, New Delhi, and is fascinated with Web 2.0. He loves to play around with Linux and will be happy to receive any queries or suggestions at *rahulgupta172@yahoo.com*.